



MÓDULO M8

SERVERLESS Y CONTENEDORES (DOMINIO 3)





MÓDULO M8

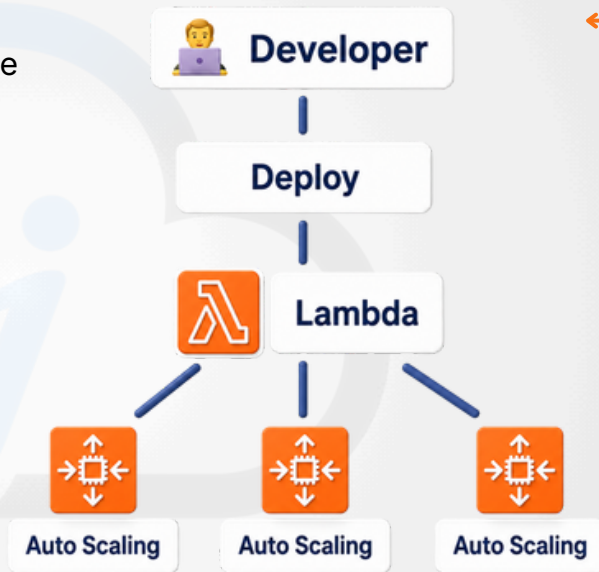
Sesión 8.1 — ¿Qué es Serverless?



SERVERLESS



- **Serverless** es un nuevo paradigma en el que los desarrolladores **ya no tienen que gestionar servidores** — solo despliegan código.
- Solo despliegan **funciones**: pequeñas unidades de lógica ejecutadas **bajo demanda**.
- Inicialmente **Serverless == FaaS** (Function as a Service). Hoy **abarca todo lo gestionado**: bases de datos, mensajería, almacenamiento, etc.



«No hay servidores que gestionar» / «don't provision servers»

SERVERLESS



- **AWS Lambda** fue el pionero del modelo FaaS en la nube pública.
- Serverless **no significa que no haya servidores** — significa que simplemente **no los ves, ni gestionas ni aprovisionas**.
- Ejemplos de servicios serverless en AWS: **Lambda, Fargate, Amazon S3, DynamoDB, API Gateway, SNS, SQS**.



💡 Serverless ≠ gratis: sigues pagando por uso del servicio.

CHECK YOURSELF?



Un colega dice «con serverless no hay servidores». ¿Es correcto?



¿Cuál fue el servicio AWS pionero del modelo **FaaS** (Function as a Service)?

ANSWERS



✓ **No exactamente.** Los servidores existen — los gestiona AWS. Tú no los ves, aprovisiones ni parcheas. Serverless = sin servidores bajo **tu** control.



✓ **AWS Lambda.** Lanzado en 2014, fue el primero en popularizar el modelo de ejecución de funciones bajo demanda sin gestionar infraestructura.





MÓDULO M8

Sesión 8.2 — AWS Lambda — El modelo



LAMBDA

- Es un servicio de **cómputo sin servidores** que permite ejecutar código sin crear ni administrar instancias **EC2**.
- Funciona por **eventos**: por ejemplo, se puede ejecutar cuando alguien **sube un archivo a S3**, llega una **petición a una API** o cambia un dato en una base de datos.
- AWS se encarga automáticamente de la **infraestructura**, el **escalado**, la **disponibilidad** y el mantenimiento del entorno.
- Se paga por el **tiempo de ejecución** del código y la cantidad de **invocaciones**, no por tener servidores encendidos todo el tiempo.



«milliseconds» + «event-driven» + «no servidor que administrar» ⇒ Lambda casi siempre.



Jemjaf



EC2 VS LAMBDA



AMAZON EC2



- Servidores virtuales en el Cloud.
- **RAM y CPU.**
- Funcionamiento continuo (pagas **aunque no haga nada**).
- Debes **intervenir**: añadir/quitar instancias

LAMBDA



- **Funciones virtuales** — sin servidores que gestionar
- **Tiempo** — ejecuciones cortas (máx. 15 min)
- **Bajo demanda** — solo cuando hay un evento
- **Automatizado** — Lambda escala solo



Jemjaf

BENEFICIOS DE AWS LAMBDA

- **Precios sencillos:** paga por **solicitud** (request) + **tiempo de cómputo** en milisegundos.
- **Free Tier (capa gratuita):** **1.000.000 de solicitudes** + **400.000 GB-segundos** de cómputo al mes — gratis para siempre.
- Integrado con **todo el ecosistema AWS** (S3, DynamoDB, EventBridge, SQS, SNS, etc.).
- **Event-driven (dirigido por eventos):** las funciones se invocan automáticamente cuando ocurre un evento.
- **Múltiples lenguajes** soportados.
- Monitorización integrada con **Amazon CloudWatch**.
- Hasta **10 GB de RAM** por función — y más RAM también mejora **CPU** y **red** automáticamente.



LENGUAJES SOPORTADOS

- Node.js (JavaScript), Python, Java, C# (.NET), Go, Ruby, PowerShell.
- **Custom Runtime API** (API de runtime personalizado): soporta la comunidad — ej. **Rust**.
- **Container images (Docker)**: la imagen debe implementar la Lambda Runtime API. Para Docker arbitrario se prefiere **ECS / Fargate**.



CHECK YOURSELF?



Un desarrollador sube una función Lambda con **256 MB de RAM** y luego la sube a **1 GB**. ¿Qué más mejora **automáticamente** además de la RAM?



¿Cuántas solicitudes incluye el **Free Tier** de AWS Lambda cada mes?

ANSWERS



✓ **CPU y red.** En Lambda, aumentar la RAM también incrementa proporcionalmente la CPU y el ancho de banda de red asignado a la función.



✓ **1.000.000 de solicitudes** (más 400.000 GB-segundos de tiempo de cómputo). Esto es permanente, no solo los primeros 12 meses.





MÓDULO M8

Sesión 8.3: AWS Lambda — Casos de uso, Precios y Amazon API Gateway



LAMBDA EN ACCIÓN + API GATEWAY

CASO DE USO 1 — BIENVENIDA AUTOMÁTICA AL USUARIO NUEVO (EVENT-DRIVEN EMAIL)



- Usuario se registra → evento dispara función Lambda → Lambda envía correo de bienvenida al nuevo usuario (vía **Amazon SES** / Simple Email Service).
- El código **solo ejecuta cuando hay un registro real** — no pagas cómputo en espera.
- Ejemplo perfecto de arquitectura **event-driven**: el evento (registro) dispara la acción (correo), sin servidores propios.



LAMBDA EN ACCIÓN + API GATEWAY

CASO DE USO 2 — TAREA PROGRAMADA (SCHEDULED JOB) SERVERLESS

- Amazon EventBridge → regla **scheduled** (programada) «cada día a las 7:00 AM» → función Lambda verifica que los procesos críticos de la base de datos se están ejecutando correctamente → alerta si algo falla.
- Reemplaza los cron jobs tradicionales sin mantener una instancia EC2 encendida las 24 h solo para lanzar un chequeo diario.
- Vocabulario clave del examen: «**scheduled**», «**programming**», «**every day at**», «**recurring task**» → patrón EventBridge + Lambda.



PRECIOS DE AWS LAMBDA (EJEMPLO)



- **Pago por llamadas (invocaciones):**
 - Primeras **1.000.000 solicitudes**: GRATIS
 - Luego: **\$0.20 por millón** de solicitudes (\$0.0000002 por solicitud)
- **Pago por duración (en incrementos de 1 ms):**
 - **400.000 GB-segundos** al mes: GRATIS
 - Equivale a 400.000 s con función de 1 GB RAM, o 3.200.000 s con función de 128 MB RAM
 - Luego: **\$1 por 600.000 GB-segundos**
- **Resultado: ejecutar Lambda es muy barato**
— ideal para cargas de trabajo esporádicas.



AMAZON API GATEWAY



- Servicio **totalmente gestionado** para que los desarrolladores puedan crear, publicar, mantener, monitorear y asegurar APIs fácilmente.
- **Serverless y escalable** — no gestionas servidores de API.
- Soporta **APIs RESTful** y **APIs WebSocket** (tiempo real bidireccional).
- Incluye: seguridad, autenticación de usuarios, claves de API (API keys), monitoreo, throttling.
- **Patrón CLF más repetido:** Cliente → **API Gateway** → **Lambda** → **DynamoDB** (backend HTTP serverless completo).



💡 API Gateway + Lambda = backend HTTP serverless típico en ítems **CLF-C02**.

CHECK YOURSELF?



Una empresa quiere ejecutar un chequeo de salud en su base de datos **cada día a las 7:00 AM** de forma automática y sin mantener servidores encendidos. ¿Qué arquitectura serverless usar?



¿Qué servicio AWS permite **exponer una función Lambda como endpoint HTTP/REST** de forma serverless?

ANSWERS



✓ **Amazon EventBridge** (regla **scheduled**) → **AWS Lambda**. EventBridge dispara la función Lambda según el horario programado. No se necesita una instancia EC2 corriendo 24x7 solo para lanzar un job diario.



✓ **Amazon API Gateway**. Actúa como puerta de entrada gestionada para las APIs: maneja autenticación, throttling, SSL y dirige las solicitudes a Lambda sin que el cliente sepa qué hay detrás.





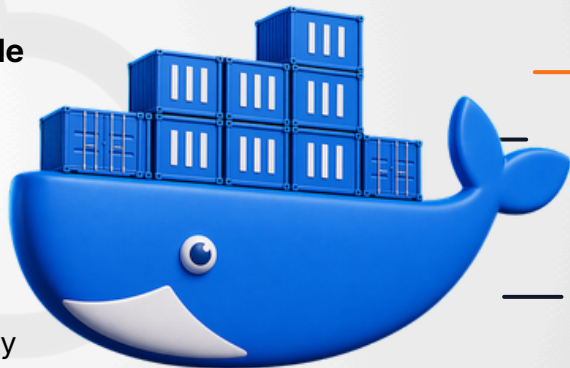
MÓDULO M8

Sesión 8.4 — Contenedores — Conceptos base



¿QUÉ ES DOCKER?

- Docker es una tecnología de código abierto que permite a los desarrolladores **empaquetar aplicaciones** y todas sus **dependencias** (bibliotecas, configuraciones) en **contenedores ligeros y portátiles**.
- **Las aplicaciones se ejecutan igual, independientemente de dónde se ejecuten**
 - Cualquier máquina
 - No hay problemas de compatibilidad
 - Comportamiento predecible
 - Menos trabajo
 - Más fácil de mantener y desplegar
 - Funciona con cualquier lenguaje, cualquier sistema operativo y cualquier tecnología
- Amplía y reduce los contenedores muy rápidamente (en segundos)



aws

CONTENEDORES VS MÁQUINAS VIRTUALES



- **Container image:** plantilla inmutable; **task** ejecuta proceso aislado vía tecnología del núcleo.
- vs **Máquina Virtual (VM)** alojada en **hypervisor** (hipervisor): los **contenedores comparten kernel** del SO anfitrión ⇒ más livianos; la **VM aísla** un SO invitado completo.
- **Docker** aparece como ecosistema de referencia práctica (el exam no hace Dockerfile profundo pero sí vocabulario.)

💡 Contenedor con todo: el contenedor lleva la app + sus dependencias, y funciona igual en cualquier entorno sin reempacar nada.

CHECK YOURSELF?



¿Un contenedor **Docker** requiere un sistema operativo invitado completo (guest OS) como una **VM**?



En una sola frase, ¿cuál es la ventaja clave de los contenedores en términos de consistencia entre entornos?

ANSWERS



✓ **No.** Los contenedores **comparten el kernel del SO anfitrión** (host OS kernel), haciéndolos más ligeros. Las **VMs** virtualizan un SO invitado completo sobre un **hipervisor** (hypervisor).



✓ **Portabilidad consistente:** el contenedor empaqueta código + dependencias y corre igual en cualquier entorno (local, dev, staging, producción) eliminando el problema «funciona en mi máquina».





MÓDULO M8

Sesión 8.5 — Kubernetes, Amazon ECS, Amazon EKS y AWS Fargate



KUBERNETES

- **Kubernetes** (K8s para los amigos) es un sistema de **código abierto** (open source) para la **orquestación** de contenedores: administra, despliega y escala aplicaciones en contenedores de forma automática.
- **Orquestación** = alguien que **coordina** cuántos contenedores corren, dónde, cuándo escalan y qué pasa si uno falla — **igual que un director de orquesta** coordina a todos los músicos.



💡 Kubernetes = orquestación open source; vocabulario: pods, manifests, control plane.
Agnóstico de nube.

KUBERNETES

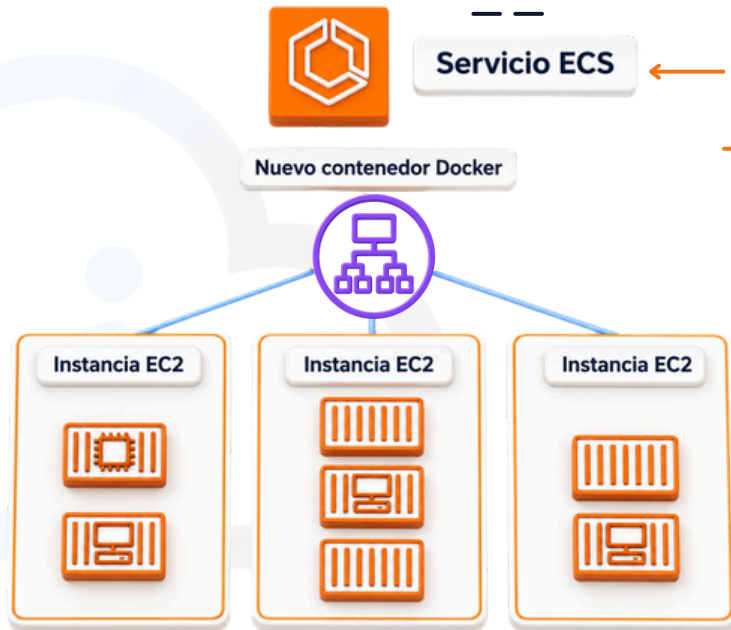


- Kubernetes es **agnóstico de la nube** (cloud-agnostic): funciona en AWS, Azure, GCP o en tu propio servidor.
- Sus unidades de ejecución se llaman **pods**: un pod agrupa uno o más contenedores que comparten red y almacenamiento.

aws

AMAZON ECS

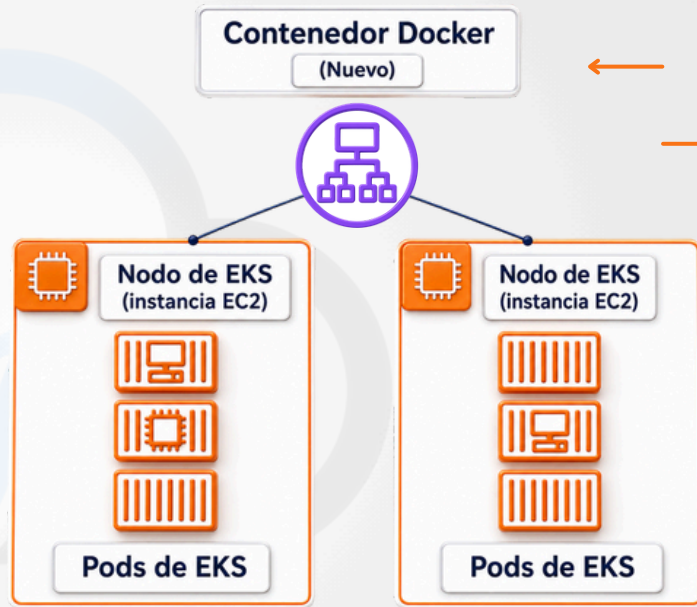
- Amazon ECS (Elastic Container Service): orquestador de contenedores nativo de AWS.
- Vocabulario propio: **Tasks** (tareas) y **Services** (servicios). No es Kubernetes.
- Tiene integraciones con el Application Load Balancer.



💡 ECS = orquestador nativo AWS — vocabulario Task / Service. No es K8s.

AMAZON EKS

- **Amazon EKS (Elastic Kubernetes Service):** Kubernetes administrado en AWS. AWS gestiona el control plane; tú traes tus manifests y pods existentes. Para equipos que ya usan K8s.
- Los contenedores pueden alojarse en:
 - **Instancias EC2**
 - **Fargate** (Serverless)

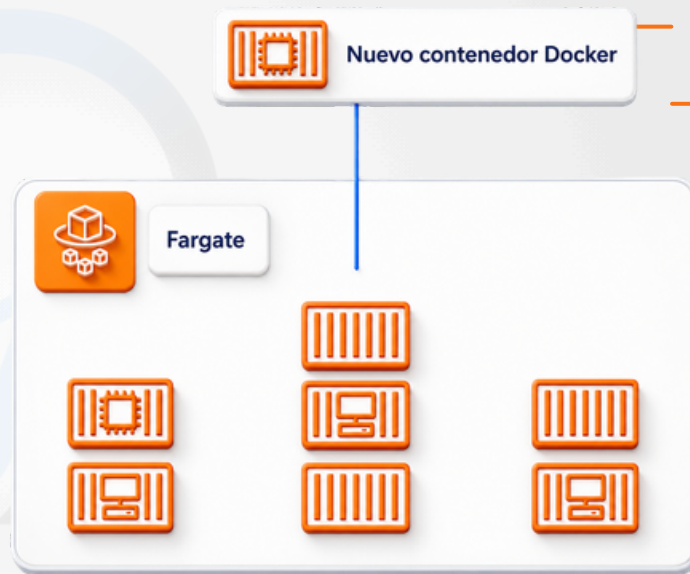


💡 EKS = Kubernetes administrado por AWS — para quienes ya usan K8s.

AWS FARGATE



- **AWS Fargate:** motor de cómputo **serverless** para contenedores — funciona tanto con **ECS** como con **EKS**.
- No seleccionas ni gestionas instancias EC2 del clúster; AWS provee el cómputo abstracto.
- **Oferta serverless**



💡 Fargate = «sin instancias EC2 del clúster bajo tu control» — aplica a ECS y EKS.

¿DÓNDE CORREN FÍSICAMENTE LOS CONTENEDORES?



- **EC2 Launch Type:** tú aprovisionas instancias **EC2** y los contenedores se ejecutan dentro de esas instancias.
- **Fargate:** AWS aprovisiona el cómputo por ti — tú solo defines qué contenedores necesitas y cuántos recursos requieren.

 Opción	 ¿Quién pone las EC2?	 Tú mantienes	 AWS mantiene
 ECS / EKS con EC2 Launch Type	Tú — aprovisionas y parchas las instancias	AMI, parches, tamaño de instancia	infraestructura subyacente AWS
 ECS / EKS con Fargate	AWS — invisible para ti	Definición de task/pod y contenedor	toda la infraestructura del clúster

CHECK YOURSELF?



¿Cuál es la diferencia entre **ECS** y **EKS** en una oración?



Un equipo quiere correr contenedores en AWS **sin gestionar las instancias EC2 del clúster**. ¿ECS con EC2 Launch Type o Fargate?



¿En qué se diferencia **Kubernetes** de **Docker**?

ANSWERS



✓ **ECS** es el orquestador de contenedores **nativo de AWS** (vocabulario Tasks/Services). **EKS** es **Kubernetes administrado** por AWS — para equipos que ya usan K8s y quieren mantener esa plataforma en la nube.



✓ **Fargate**. Con Fargate, AWS aprovisiona y gestiona el cómputo subyacente; el equipo solo define la tarea (task) y el contenedor. No hay instancias EC2 del clúster que administrar, parchear ni redimensionar.



✓ **Docker** empaqueta y corre contenedores individuales. **Kubernetes** orquesta múltiples contenedores a escala: decide dónde corren, cuántos hay, cómo escalan y qué pasa si uno falla. Son complementarios, no competidores.





MÓDULO M8

Sesión 8.6: Banco de práctica



PREGUNTA 1



¿Cuál es el **modelo de facturación** de **AWS Lambda**?

- A) Por hora de instancia en ejecución, igual que EC2
- B) Por GB de almacenamiento usado en S3
- C) Por milisegundos de ejecución más el número de invocaciones
- D) Tarifa fija mensual independiente del uso

PREGUNTA 1



¿Cuál es el *modelo de facturación* de **AWS Lambda**?

✓ Respuesta correcta: **C** 💡 Lambda cobra por **duración de ejecución en milisegundos** + número de **invocaciones**. Si la función no ejecuta, no pagas cómputo. Este modelo es radicalmente diferente a EC2 (por hora/segundo) o S3 (por GB almacenado).

PREGUNTA 2



Un desarrollador necesita procesar un archivo de análisis que tarda **20 minutos** en completarse. Decide usar **AWS Lambda**. *¿Qué ocurrirá?*

- A) Lambda procesa sin límite de tiempo si la memoria es suficiente
- B) Lambda dividirá automáticamente el trabajo en múltiples invocaciones paralelas
- C) La función alcanzará el límite de timeout (máximo 15 minutos) y será terminada
- D) Lambda aumentará temporalmente su timeout para esta invocación específica

PREGUNTA 2



Un desarrollador necesita procesar un archivo de análisis que tarda **20 minutos** en completarse. Decide usar **AWS Lambda**. ¿Qué ocurrirá?



Respuesta correcta: **C**



Lambda tiene un **timeout máximo de 15 minutos** (900 segundos). Una tarea de 20 minutos superará ese límite y será **terminada forzosamente**. Para procesos de larga duración, considerar **AWS Fargate** (contenedor sin límite de 15 min) o **AWS Batch**.



Jemjaf

PREGUNTA 3



¿Cuál es la **diferencia principal** entre un contenedor Docker y una máquina virtual (VM)?

- A) Los contenedores requieren un hipervisor; las VMs no
- B) Los contenedores comparten el kernel del SO anfitrión y son más ligeros; las VMs virtualizan un SO invitado completo
- C) Los contenedores son más lentos porque empaquetan más software que las VMs
- D) Las VMs son más portables que los contenedores entre entornos

PREGUNTA 3



¿Cuál es la **diferencia principal** entre un contenedor Docker y una máquina virtual (VM)?



Respuesta correcta: B



Los contenedores **comparten el kernel del SO anfitrión** y arrancan en segundos, con menor overhead de memoria. Las VMs incluyen un **SO invitado completo** virtualizado por un hipervisor — más aislamiento pero más pesadas. Para CLF-C02: contenedor = «proceso aislado ligero»; VM = «computadora dentro de una computadora».



Jemjaf

PREGUNTA 4



Un equipo usa **Kubernetes** en producción on-premises y quiere migrar a AWS manteniendo su plataforma de orquestación. *¿Qué servicio elegir?*

- A) Amazon ECS con Fargate
- B) AWS Lambda para todos los microservicios
- C) Amazon EKS (Elastic Kubernetes Service)
- D) Amazon ECS con EC2 Launch Type



PREGUNTA 4



Un equipo usa **Kubernetes** en producción on-premises y quiere migrar a AWS manteniendo su plataforma de orquestación. *¿Qué servicio elegir?*



Respuesta correcta: C



Amazon EKS es **Kubernetes administrado** en AWS. AWS gestiona el control plane (plano de control) de Kubernetes; tú traes tus manifests y workloads existentes. **ECS** es el orquestador nativo de AWS con vocabulario diferente (Tasks, Services), no Kubernetes.



Jemjaf

PREGUNTA 5



Una empresa quiere ejecutar contenedores en AWS **sin gestionar ni parchear las instancias EC2** del clúster de contenedores. *¿Qué opción de cómputo usar?*

- A) Amazon ECS con EC2 Launch Type
- B) Amazon EC2 con Auto Scaling Group
- C) AWS Fargate
- D) Amazon EKS con instancias reservadas

PREGUNTA 5



Una empresa quiere ejecutar contenedores en AWS **sin gestionar ni parchear las instancias EC2** del clúster de contenedores. *¿Qué opción de cómputo usar?*

✓ Respuesta correcta: **C** 💡 **AWS Fargate** es el motor de cómputo serverless para contenedores. Con Fargate, AWS aprovisiona, escala y parchea la infraestructura subyacente — el equipo solo define tasks y contenedores. Con **EC2 Launch Type**, el equipo gestiona las instancias del clúster.



PREGUNTA 6



¿Cuándo es más apropiado usar **AWS Fargate** en lugar de **AWS Lambda**?

- A) Para ejecutar una función simple que responde a un evento y termina en segundos
- B) Para ejecutar un servicio web de larga duración (contenedor que corre continuamente)
- C) Para procesar imágenes individuales cuando se suben a S3
- D) Para automatizaciones que corren en milisegundos

PREGUNTA 6



¿Cuándo es más apropiado usar **AWS Fargate** en lugar de **AWS Lambda**?



Respuesta correcta: **B**



Fargate es ideal para contenedores de **larga duración** (servicios web, APIs, workers continuos) sin el límite de 15 minutos de Lambda. **Lambda** es para funciones cortas y orientadas a eventos. Criterio: si el proceso necesita más de 15 minutos, o corre como servicio continuo → **Fargate**.



PREGUNTA 7



Una empresa usa **EC2, Fargate y Lambda** con uso estable. Quieren **descuento por compromiso** con flexibilidad entre los tres servicios. *¿Qué tipo de Savings Plans incluye los tres?*

- A) EC2 Instance Savings Plans
- B) Compute Savings Plans
- C) Standard Reserved Instances
- D) Spot Instances



PREGUNTA 7



Una empresa usa **EC2, Fargate y Lambda** con uso estable. Quieren **descuento por compromiso** con flexibilidad entre los tres servicios. *¿Qué tipo de Savings Plans incluye los tres?*



Respuesta correcta: B



Compute Savings Plans aplican a **EC2, AWS Fargate y AWS Lambda** con flexibilidad de familia, región y servicio. Los **EC2 Instance Savings Plans** solo aplican a una familia de instancias EC2 específica. Las **Reserved Instances** no cubren Lambda ni Fargate.



Jemjaf

PREGUNTA 8



¿Cuáles de los siguientes son **servicios de cómputo** en los que el cliente **no aprovisiona ni administra EC2/servidores** para ejecutar cargas de trabajo? (Seleccione DOS.)

- A) Amazon EC2
- B) AWS Lambda
- C) Amazon ECS con EC2 Launch Type
- D) AWS Fargate
- E) Amazon EKS con instancias autoescaladas



PREGUNTA 8



¿Cuáles de los siguientes son **servicios de cómputo** en los que el cliente **no aprovisiona ni administra EC2/servidores** para ejecutar cargas de trabajo? (Selecione DOS.)

✅ Respuesta correcta: **B y D** 💡 **Lambda y Fargate** encajan en serverless compute: no eliges instancias EC2 del cliente. **EC2, ECS con EC2 y EKS con nodos EC2** implican **gestionar** instancias. (Otros servicios serverless como **Kinesis Firehose** son de **ingesta/analytics**, no cómputo de tu app — por eso esta pregunta acota «cómputo».)



PREGUNTA 9



Una emprendedora quiere lanzar un blog **WordPress** rápidamente con un **precio plano y predecible** desde menos de \$5/mes y sin necesidad de configurar VPC, EBS ni instalar WordPress manualmente. *¿Qué servicio AWS satisface mejor estos requisitos?*

- A) Amazon EC2 con AMI de Ubuntu y configuración manual
- B) AWS Elastic Beanstalk con plataforma PHP
- C) Amazon Lightsail con blueprint de WordPress
- D) AWS Lambda con un layer de PHP

PREGUNTA 9



Una emprendedora quiere lanzar un blog **WordPress** rápidamente con un **precio plano y predecible** desde menos de \$5/mes y sin necesidad de configurar VPC, EBS ni instalar WordPress manualmente. *¿Qué servicio AWS satisface mejor estos requisitos?*

✓ Respuesta correcta: **C** 💡 **Amazon Lightsail** ofrece **blueprints preconfigurados** (WordPress, LAMP, Nginx, Node.js...) con **precio plano** desde ~\$3.50/mes que incluye servidor + 20 GB SSD + transferencia. Se lanza en **3 clics**, sin configurar VPC ni EBS. EC2 requeriría todo el proceso manual; Beanstalk es PaaS para apps propias, no plantillas listas; Lambda no aloja un sitio WordPress tradicional.





MÓDULO M8

Sesión 8.5: Datos de certificación



DATOS DE CERTIFICACIÓN



Tema	Dato clave EN/ES
Serverless (paradigma)	Devs no gestionan servidores; solo despliegan código/funciones; AWS gestiona la infra
Lambda	Event-driven; billing por milisegundos + invocaciones; timeout máx 15 min ; RAM hasta 10 GB
Lambda Free Tier	1.000.000 solicitudes + 400.000 GB-segundos al mes — permanente, no solo 12 meses
Lambda casos típicos	S3 PutObject -> thumbnail; EventBridge -> CRON; API Gateway -> backend HTTP
Lambda lenguajes	Node.js, Python, Java, C#, Go, Ruby, PowerShell + Custom Runtime (Rust)
Amazon API Gateway	Expone Lambda/servicios como API REST o WebSocket ; serverless; gestiona auth, throttling
Kubernetes	Orquestación open source; vocabulario: pods , manifests ; cloud-agnostic (AWS/Azure/GCP)
Fargate	Contenedores sin gestionar EC2 del clúster ; sin límite de 15 min
EKS	Kubernetes administrado en AWS — trae tus pods y manifests existentes
ECS	Orquestador nativo AWS ; vocabulario: Tasks / Services (no es K8s)
AWS Batch	Lotes de larga duración ; cuando Lambda no alcanza por tiempo
Compute Savings Plans	Cubre EC2 + Fargate + Lambda con flexibilidad de familia/región
Serverless ≠ gratis	«You don't provision servers» pero sigues pagando por uso
Lightsail	VPS todo-en-uno; precio plano desde ~\$3.50/mes; blueprints (WordPress, LAMP); ☒ HA / ☐ sin auto-scaling



TRAMPAS FRECUENTES

- «Event-driven + duración en segundos/minutos cortos» → **Lambda** (ojo **timeout 15 min**).
- «Contenedor **corriendo como servicio** continuo / siempre encendido» → **Fargate** (no Lambda).
- «Necesito **Kubernetes** en AWS» → **EKS** (vocabulario K8s; **ECS** es otro modelo).
- «**Sin** gestionar **EC2** del clúster de contenedores» → **Fargate**, no **EC2 Launch Type**.
- «Job > 15 min o horas» → **Fargate** o **AWS Batch** — no Lambda.
- «Backend **HTTP** serverless» → **API Gateway** + **Lambda** (patrón examen).
- «**No servers**» ≠ «sin coste»: sigues pagando por uso/billing del servicio.
- «**WordPress** / principiante / precio plano ~\$3.50/mes» → **Lightsail**, no EC2 ni Beanstalk.
- **Lightsail** tiene alta disponibilidad  pero **NO auto-scaling**  — si la pregunta exige escalar automáticamente con la demanda, **no** es Lightsail.
- **Lambda Free Tier** es **permanente** (no solo 12 meses como otros servicios del AWS Free Tier).

